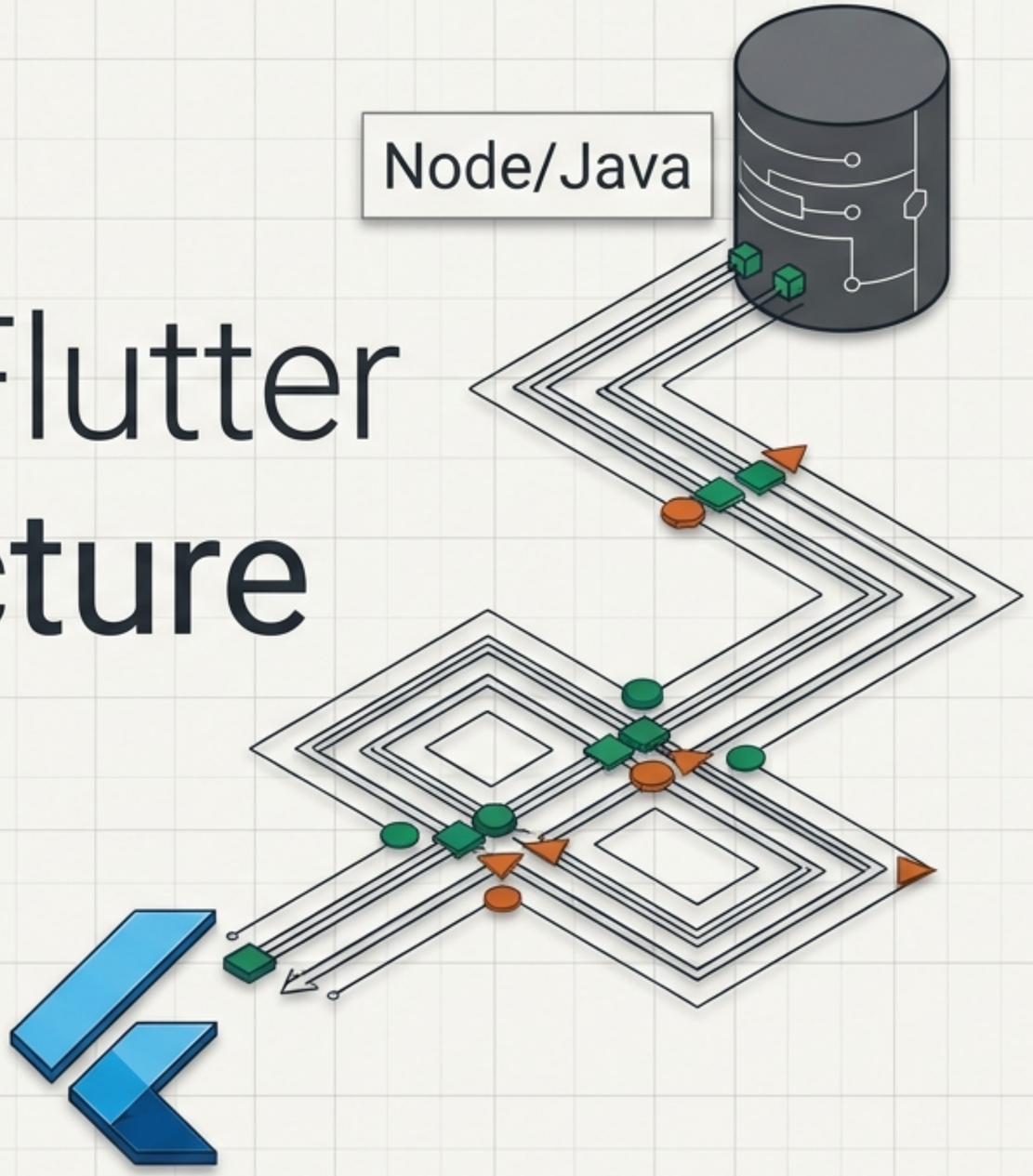
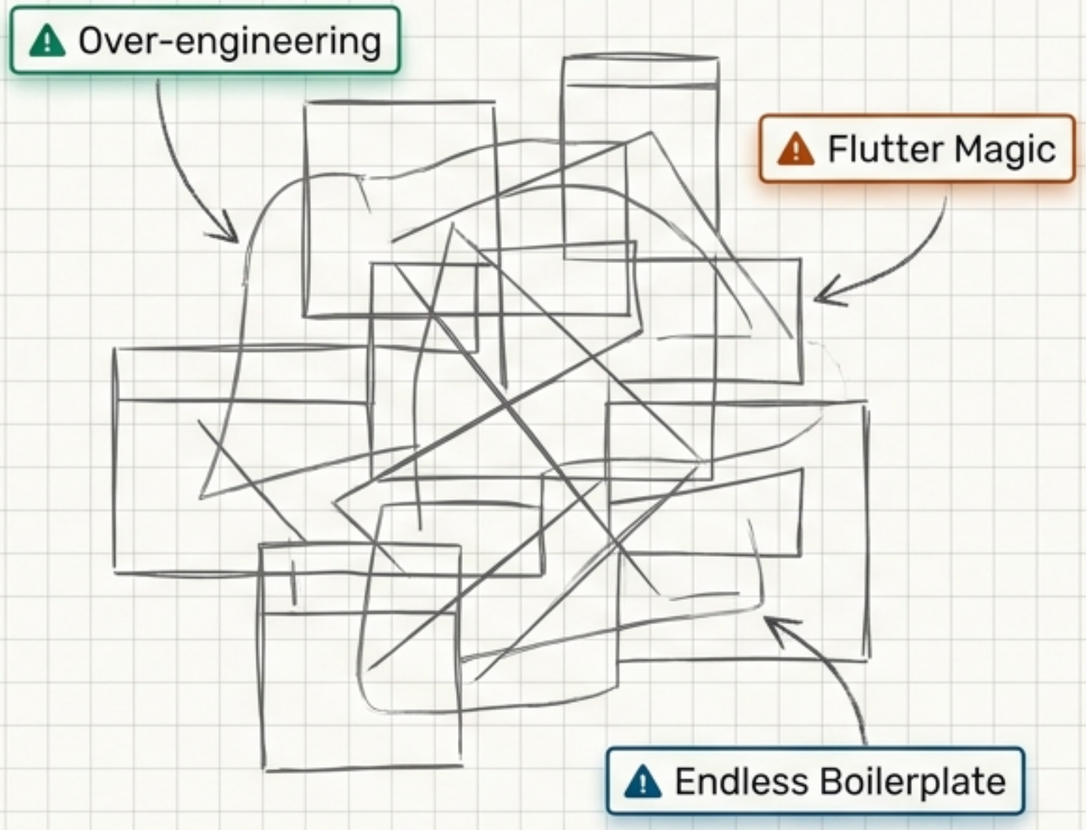
Demystifying Flutter Clean Architecture

A mental model bridge for TypeScript and Java backend developers.

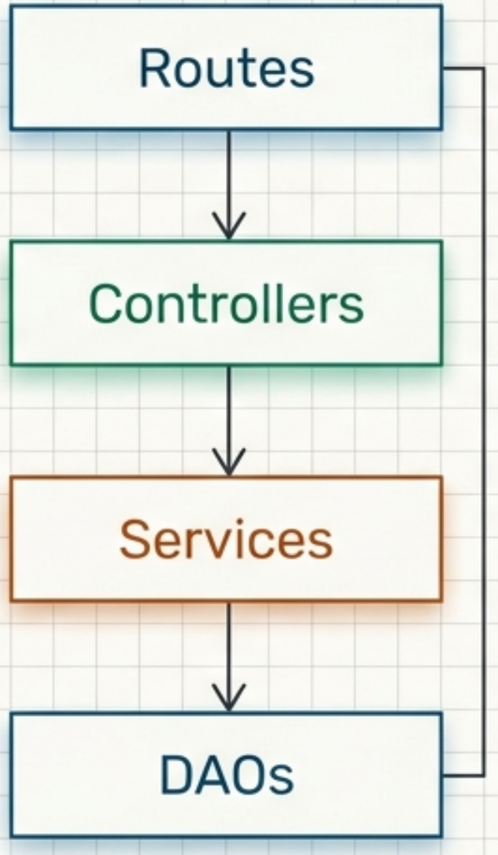


You already know Clean Architecture.

The Myth



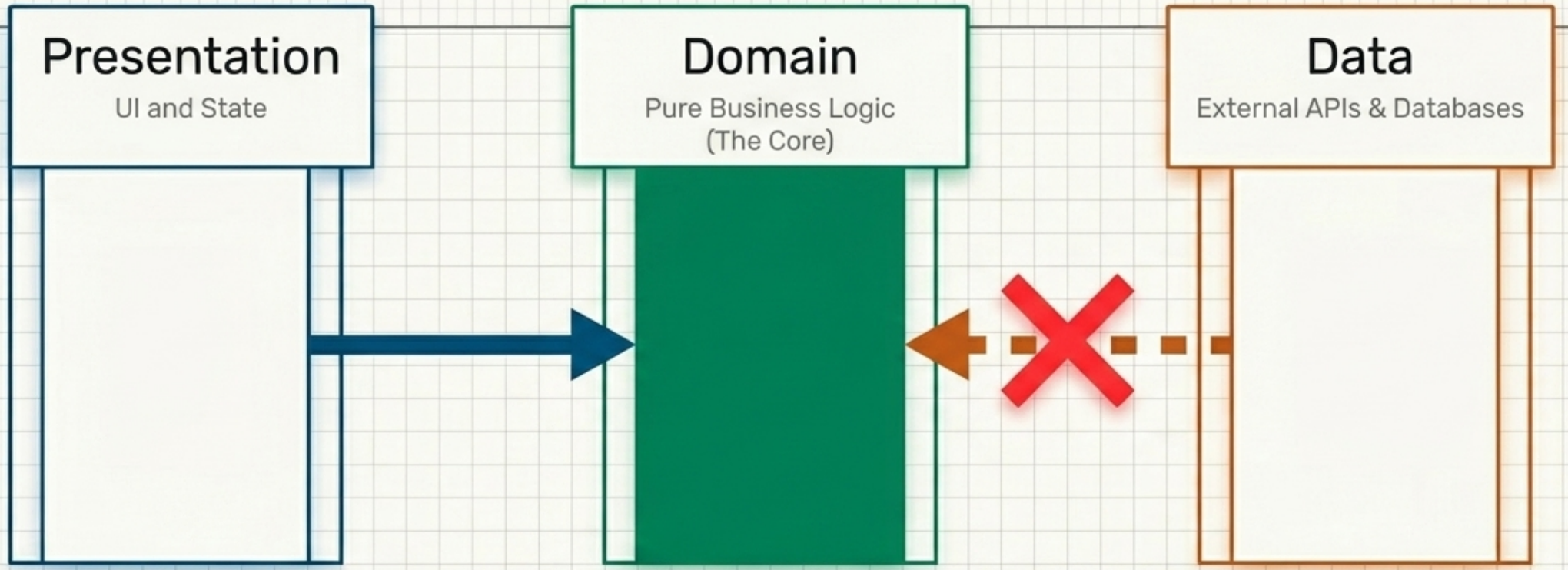
The Reality



The exact same separation-of-concerns you use every day in Express or Spring.

The only difference? It is mandatory and strictly enforced by folder structure.

Unidirectional Architectural Diagram



The Golden Rule: Arrows Only Point Inward

The Domain knows nothing else exists. It has zero knowledge of how data is fetched or rendered.

- Dependencies always point towards the center (Domain).
- Inner layers (Domain) are independent of outer layers (Presentation, Data).
- External details should not leak into the core logic.

[TS: Concepts] | [Java: Concepts]

The Rosetta Stone

Flutter Term	TypeScript / Node	Java / Spring
Entity	`type` / Zod schema	Plain Java POJO
Model (DTO)	DTO class w/ serialization	Jackson `@JsonProperty` class
Repository (Abstract)	`interface`	Java interface (The contract)
Datasource	`Axios` call / DAL	DAO (Direct DB/SDK calls)
UseCase	Controller handler / Service function	`@Service` method
Provider	Zustand store + DI container	Spring `@Autowred` + ViewModel
Screen	React component	Thymeleaf view

The Feature Blueprint

`lib/features/hello/`

Fractal Architecture

Every single feature in the app follows this exact structure.

The folder tree physically enforces the boundaries.

[TS: Zod Schema / Interface] | [Java: POJO / Interface]

The Domain Layer (Core)

`hello_message.dart (Entity)`

Concept: The **pure business object**. No Firebase, no Flutter, no JSON parsing. Just the shape of the data.

Usage: Used by UseCases, Repositories, and UI State.

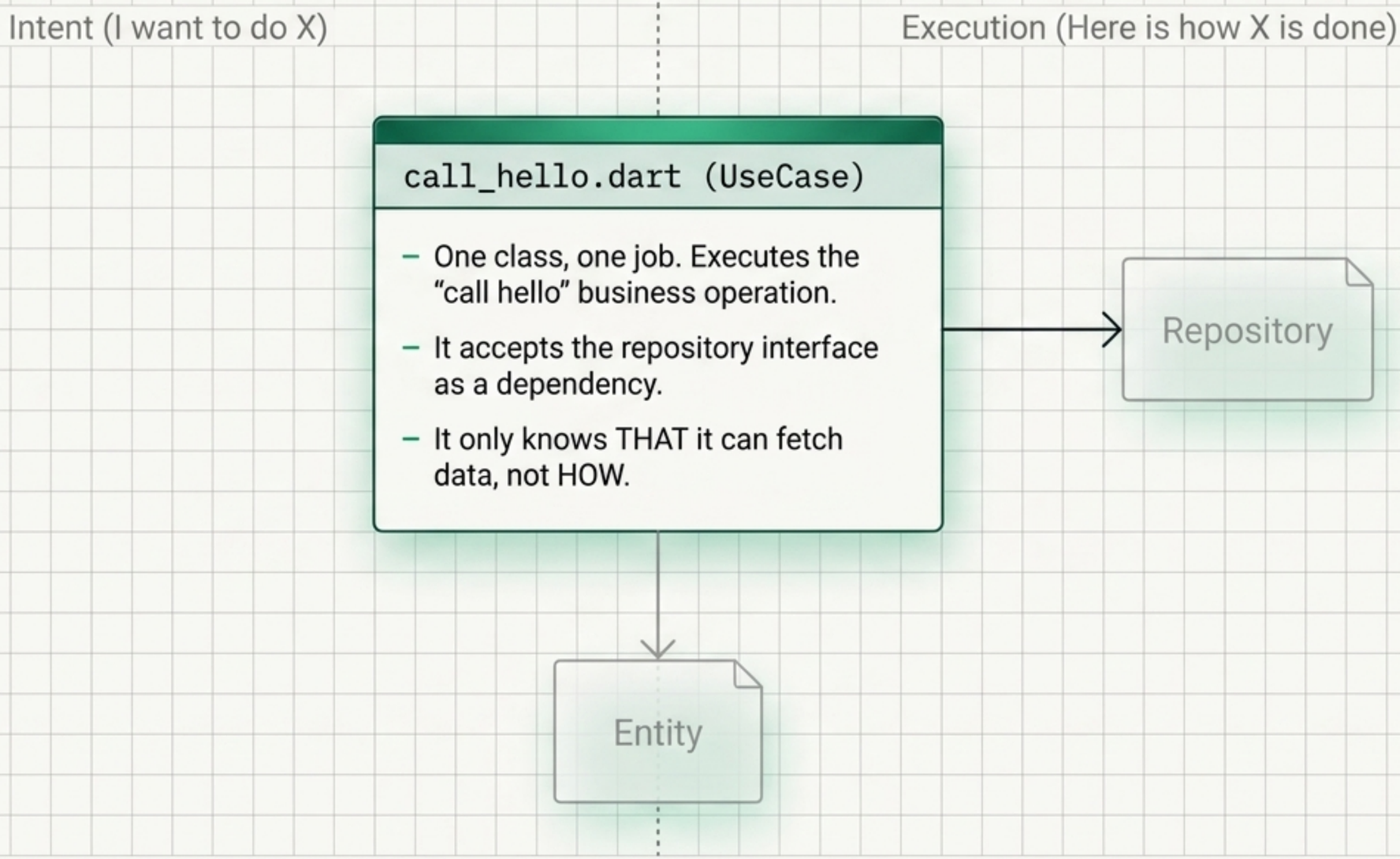
`hello_repository.dart (Abstract Repo)`

Concept: The contract. Defines WHAT operations exist (*callHello*), not HOW they work.

Usage: The Domain layer doesn't care if data comes from Firebase or a mock. It only relies on this interface.

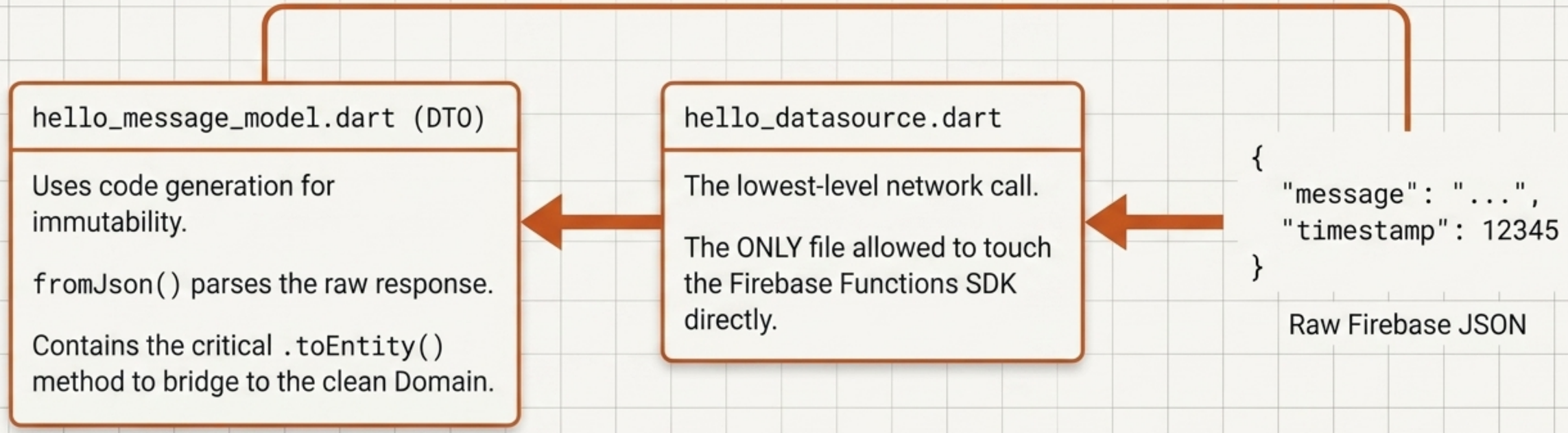
The Domain Layer (Action)

[TS: Service Function] | [Java: @Service Class]



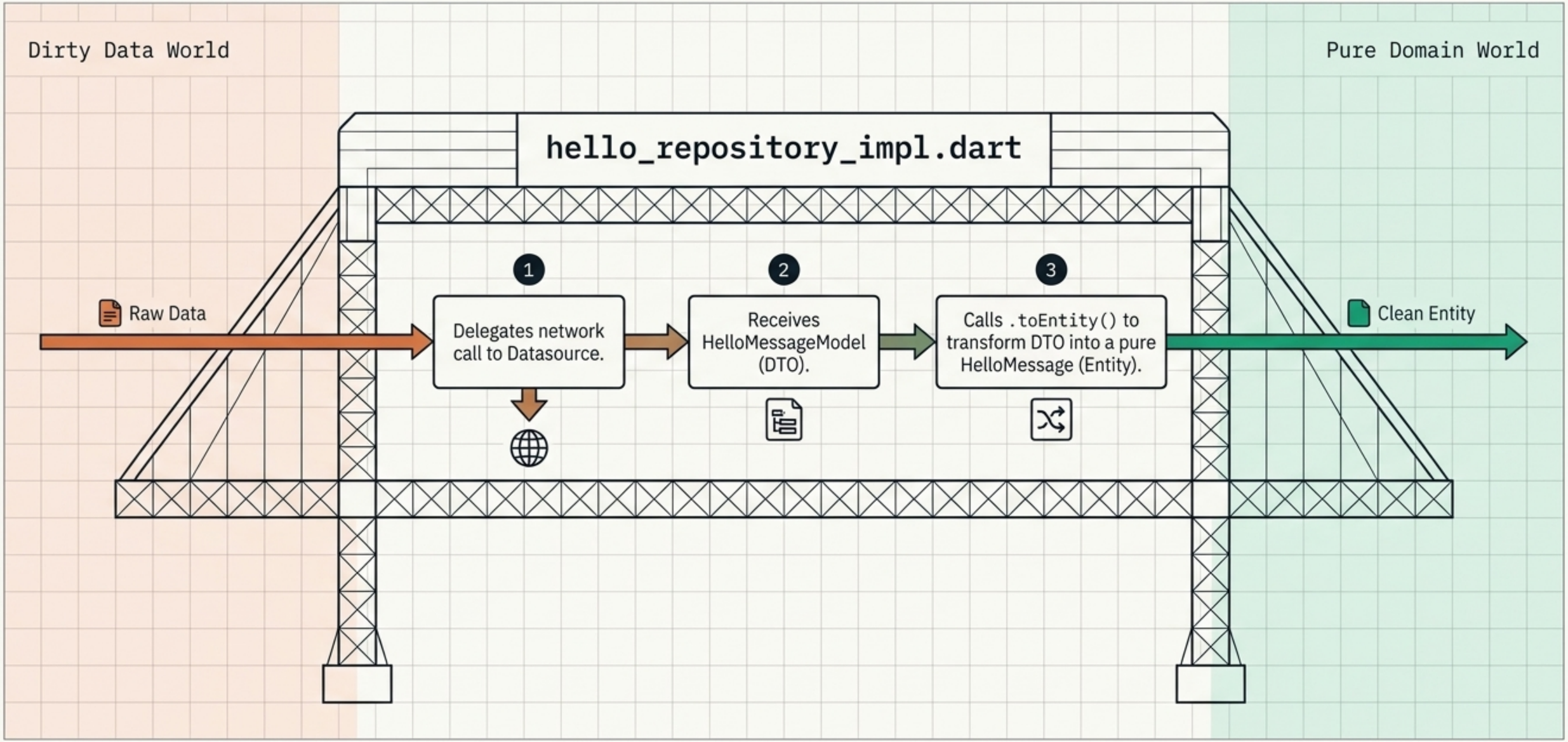
The Data Layer (Network)

`[TS: Axios Call + DTO] | [Java: DAO + Jackson DTO]`



The Data Layer (Adapter)

[TS: Class implementing Interface] | [Java: @Repository Impl]



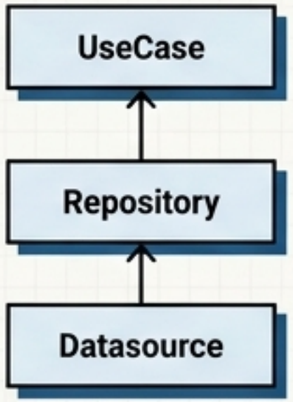
The Presentation Layer (State & DI)

[TS: Zustand + tRPC Context] |
[Java: Spring @Configuration + ViewModel]

hello_provider.dart

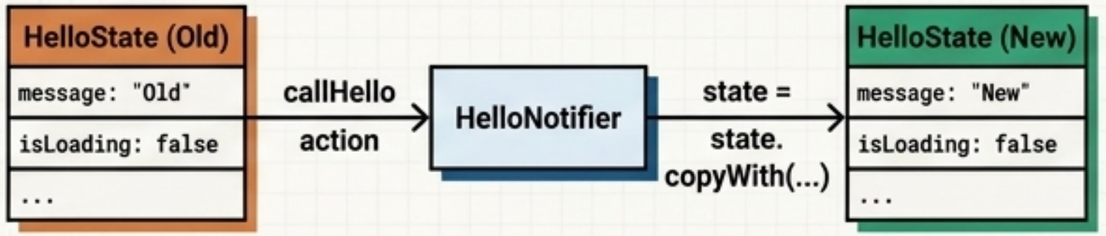
Part 1: DI Wiring

- Acts like `tsyringe` or Spring's Context.
- Builds the dependency chain: `Datasource` -> `Repository` -> UseCase.
- `ref.watch()` resolves dependencies dynamically.



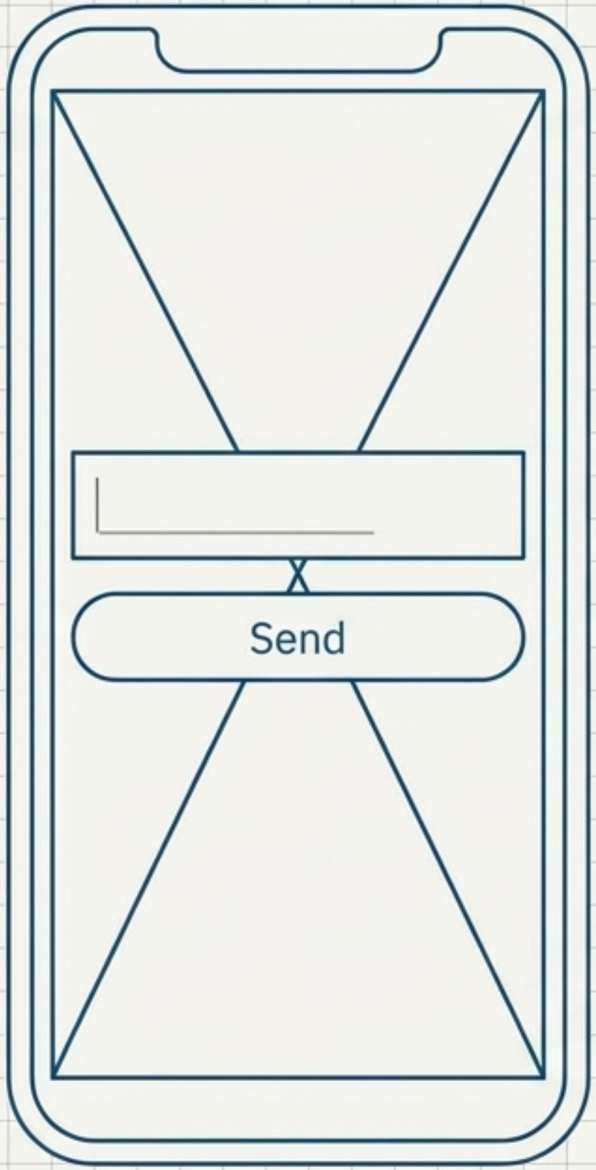
Part 2: State & Actions

- `HelloState`: Immutable state shape (like a Redux store).
- `HelloNotifier`: Holds the `callHello` action method.
- Uses `state = state.copyWith(...)` to mutate (like React setState).



The Presentation Layer (UI)

[TS: React Component] | [Java: Thymeleaf View]



hello_screen.dart

Concept: A ConsumerStatefulWidget is exactly a React component using hooks.

Reading: `ref.watch()` subscribes to state (like useSelector). Re-renders instantly when state changes.

Writing: `ref.read()` gets the notifier to fire actions (like `useDispatch`).

Purity: Knows NOTHING about Firebase or APIs. Only knows the Domain Entity and the Notifier.

Provider State

[TS: Express POST Route] | [Java: @RestController]

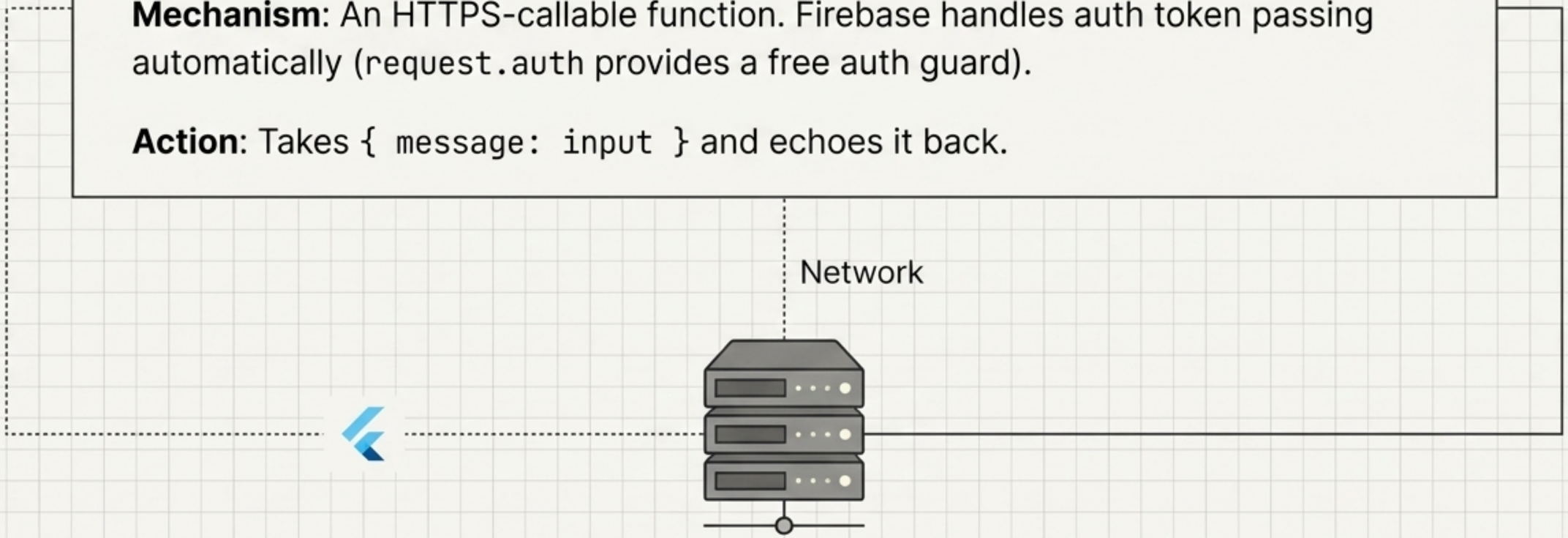
The Actual Backend

`functions/src/index.ts (The Firebase Server)`

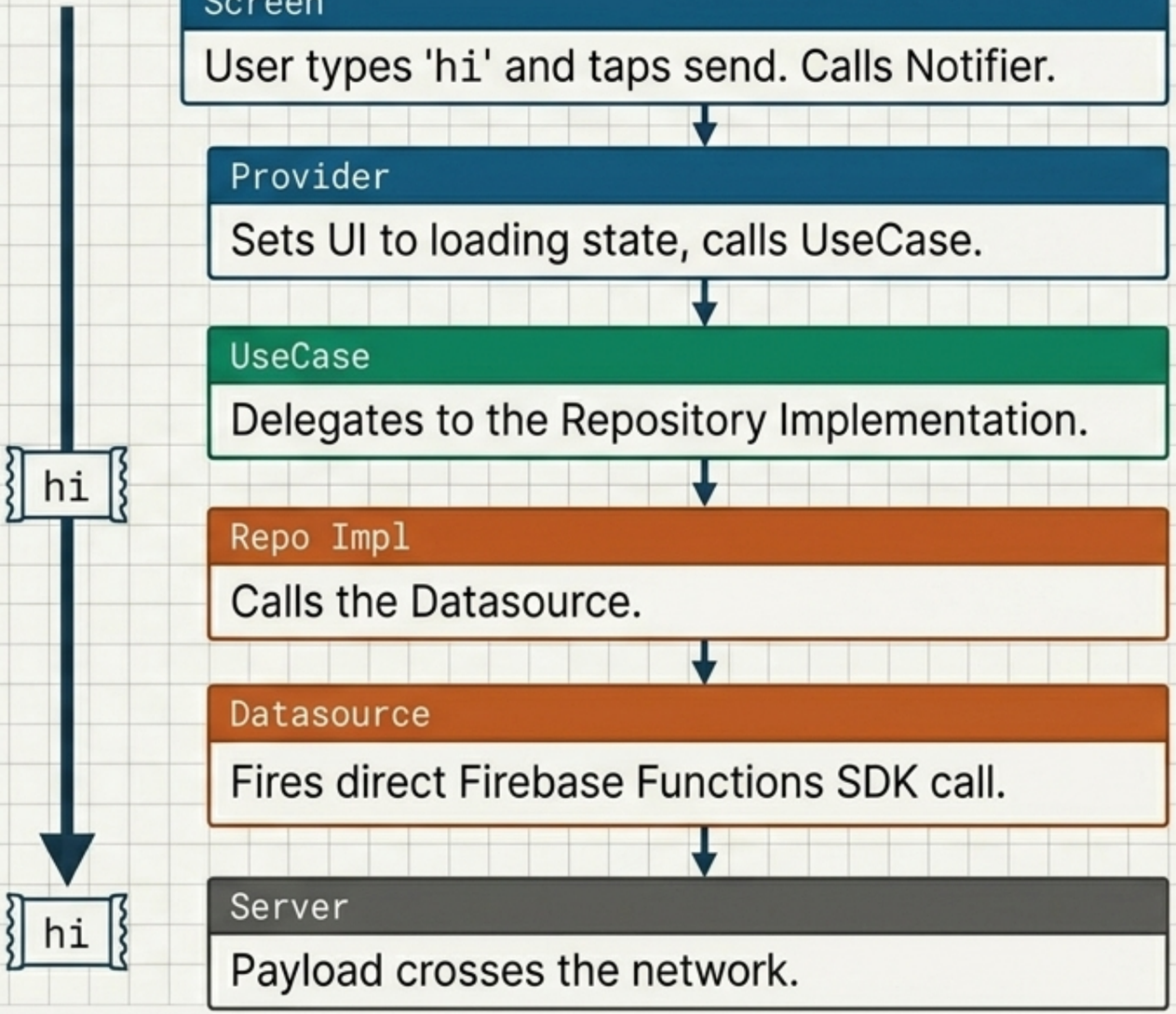
Concept: This IS a regular TypeScript backend.

Mechanism: An HTTPS-callable function. Firebase handles auth token passing automatically (`request.auth` provides a free auth guard).

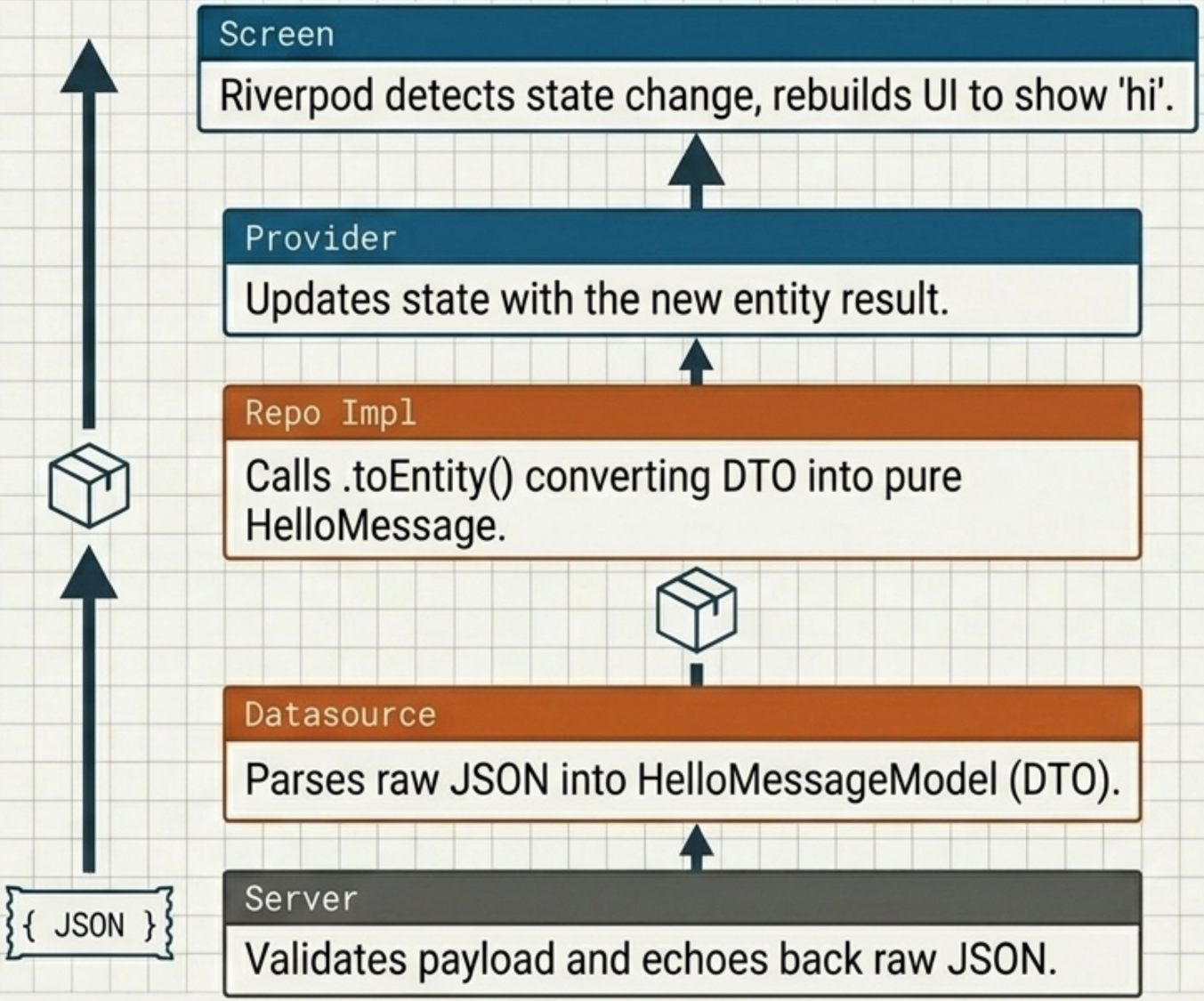
Action: Takes `{ message: input }` and echoes it back.



End-to-End Flow (Part 1: The Request)



End-to-End Flow (Part 2: The Response)



Why Bother? (The Payoff)

Without Clean Arch ❌	With Clean Arch ✅
Firebase SDK calls hidden inside UI widgets.	Firebase isolated to exactly one file (Datasource).
Must spin up live Firebase instance just to test UI.	Swap Repo Impl for a mock; test UI in 100% isolation.
Migrating from Firebase to **REST** breaks the entire app.	Only change the single **Datasource** file.

Insight: Yes, there are more files upfront. But every file is tiny, single-purpose, and independently testable—just like a properly layered backend.